Introduction à la Programmation et l'Algorithmique

CHRISTELLE CAILLOUET – CHRISTELLE.CAILLOUET@UNIV-COTEDAZUR.FR

DINO LOPEZ PACHECO – DINO.LOPEZ@UNIV-COTEDAZUR.FR

Algorithms

Problems, algorithms and computer programs

- Everything starts with a problem to solve
  - Constraints/requirements
  - Initial state (input) -> desired state (output)
  - Problem definition
- An algorithm is created to solve the problem
  - Correctness
    - with a right input, the algorithm provides de desired output
  - Efficiency
    - Measured in terms of execution duration and/or memory and/or other many other parameters
  - Complexity
    - How fast or slow is our algorithm with a given number of inputs
- Computer program
  - Actual implementation of an algorithm by means of a programming language

Example



Problem

Algorithm



Rainy Day Baking

Pizza Recipe

When the weather is bad and you are looking for indoor activities to do, cooking is a great way to keep the children entertained whilst learning new skills. A quick online search will give you lots of ideas for recipes you could make with your children, from the simple to the more complicated.

Homemade Pizzas

Who doesn't like pizza? These tasty snacks are really easy to make and, because they can choose their own toppings, will suit the taste buds of most children.

Ingredients

English muffins

Method

1. Preheat the oven to 190°C or gas mark 5.



Realization

How to create algorithms

1. Define your problem
   - How to keep the oven at a desired temperature?
2. Devise a general solution
   - Read the desired user temperature Tu. If the current temperature Tc is lower than Tu, then enable the oven heating element at its maximum capacity. If Tc is equal to Tu, provide a minimum power to the heating element. In all other cases, turn off the heating element. Wait t seconds and repeat the previous operations.

Write your algorithm

- Human phrases might be difficult to understand and then implement
- Formalize your algorithm
  - Flow chart
  - ***Pseudocode***
- Your algorithm is more important than your code to publish your solution
  - Code is subject to the language programming constraints/paradigm
  - a few programmers might know your programming language

```
Fonction lancement_four(){
  // variables du système
  Tu : température du four souhaité par l'utilisateur
(entier)
  Tf : température courante du four (entier)
  t = 60 : temps en sec avant de vérifier l'état actuel du
four (entier)

  Lire Tu
  tant que (four_est_allumé()) {
    Lire Tf
    Si (Tf < Tu){
      activer_resistance_fortement()
    } Sinon {
      Si (Tf == Tu){
        activer_resistance_faiblement()
      } Sinon {
        desactiver_resistance()
      }
    }
    attendre(t)
  }
}
```

Complexity

Algorithmic complexity

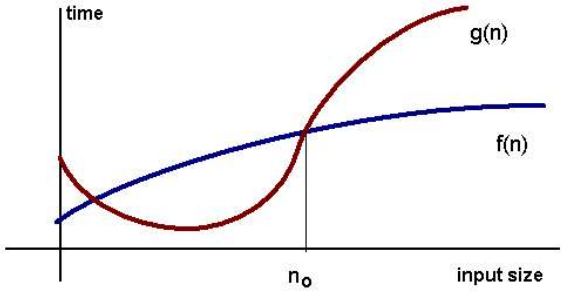
- Assess the performance of an algorithm
  - Execution time - Fast or slow
- Execution time
  - Intuitively, it depends on the characteristics of the hardware and the language programming
  - How to estimate the execution time without considering hardware or programming capabilities?
- Complexity
  - Asymptotic behavior (large input -> inf)
    - So hardware/programing language capabilities can be ignored
  - Big-O notation
  - Complexity == How fast is our algorithm
    - Complexity is not how difficult it was to design the problem solution!

Big-O

- Worst running time case
  - Upper bound

f(n) = O(g(n)) if there exists $c > 0$ and $n_0 > 0$ such that

f(n) <= c * g(n), for all $n > n_0$



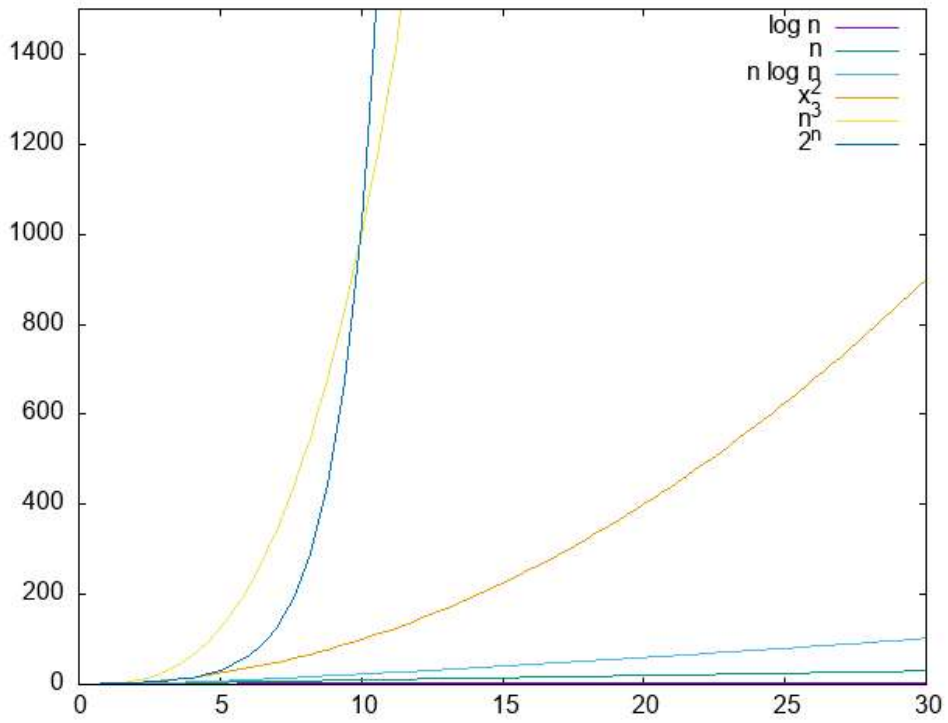
Estimating “big-Oh”

```
1. Fonction chante_allouette(partie[0..n-1]){
2.   Pour (i=0), tant que (i < n), avec (i++) {
3.     Chante « Alouette, gentille alouette »
4.     Chante « Alouette, je te plumerai »
5.     Pour (j=0), tant que (j < 2), avec (j++) {
6.       Chante « Je te plumerai » + partie[i]
7.     }
8.     Pour (j=i), tant que (j >= 0), avec (j--) {
9.       Chante « Et » + partie[j] + « ! Et » +
partie[j] + « ! »
10.    }
11.    Chante « Alouette, Alouette! »
12.    Chante « Oh, oh, oh, oh »
13.  }
14.}
```

- Ignore constant parts
  - Example : `Chante « Alouette, gentille alouette »`
- Focus on variable parts -> Input size (n) dependent
  - Segment (Line) 2 is n-dependent
  - Is line 5 n-dependent?
  - Is line 8 n-dependent?
- Estimate the cost of the variable parts
  - Segment from line 2 executes n times
  - Segment from line 8 executes n times in the worst case
  - Complexity is $O(n * n) – O(n^2)$

Big-O functions

- Simplify until you reach one of the following functions
  - O(1)
  - O(log n)
  - O(n)
  - O(n*log n)
  - $O(n^2)$
  - $O(n^k)$ -> polynomial
  - $O(a^n)$ -> exponential
- Take the biggest component
  - $n^2 + n + 3 = O(n^2)$
- Your algorithm must not have an exponential complexity!



Annex - writing a pseudocode

Il n'y a pas des règles précises ni strictes, mais en générale…

Écrire des fonctions

```
fonction somme(a,b) {
   // si malgré tout, je pense que mon pseudocode n'est pas assez claire,
   // je peux introduire des commentaires à mon algorithme, comme ici
    c = a + b ; // commentaire dans la même ligne d'une instruction
    retour c
}
```

```
fonction demande_age(nom) {
    affiche « Bienvenu. Aujourd'hui c'est le » + date()
    affiche « quel est votre âge » + nom + « ? »
    lire age ; // « age » -> variable contenant âge rentrée par l'utilisateur
}
```

Bloques conditionnels et “switch”

```
si (conditions) {
    …
} sinon { ; // sinon est optionnel
    …
}
```

```
si (conditions) {
    …
} sinon si (conditions) {
    …
} ; // sinon ou rien, au besoin
```

```
Si (var) égal à
    « a ») { … faire quelque chose}
    « b ») { … faire autre chose}
    Autre chose {… comportement par défaut}
Fin si
```

boucles

```
Faire tant que (conditions) {
    …  ;  //  chose  utile  pour  ma
fonction
}
```

```
Faire {
…
} tant que (conditions)
```

```
Pour (variable) allant de (début) à
(fin) {
 }
```

Exemple :

```
pour i allant de 0 à n {
  …
  i = i + 2 ; // on augmente la
valeur de i de 2 unités
  // si i augmentait d'une unité, on
aurait pu omettre la ligne
  // i = i + 1
}
```

Tableaux

```
tab = {} ;// tableau vide
tab = {"a","b","c"} ; // tab[0] sera égal à « a »
tab = {{1,"a"},{2,"b"},{3,"c"}} ; // tab[2,0] sera égal à « 3 »
tab1 = tab[2] ;// équivalent à tab1 = {3,"c"}
tab[2,0]=4 ; // on a changé « 3 » par « 4 ».
```